

# Agent Phantom Recovery

## 08-folder-structure.md

### Production Folder Structure

Version: 1.0

---

## 1. Purpose

This document defines the production-grade folder structure for Agent Phantom Recovery.

The structure is designed to support:

- Long-horizon agent execution
- Multi-LLM architecture
- Tool orchestration
- Memory systems
- Repository analysis
- Browser integration
- Terminal integration
- Verification pipelines
- Production deployment
- Future scalability

The structure follows separation-of-concerns principles so that each subsystem can evolve independently.

---

## 2. High-Level Repository Structure

```
agent-phantom-recovery/  
|  
├─ frontend/  
├─ backend/  
├─ infrastructure/  
├─ ai/  
├─ tools/  
├─ memory/  
├─ repository-analysis/  
├─ execution/  
├─ shared/  
└─ scripts/
```

```
| docs/
| tests/
| configs/
| deployments/
| datasets/
| logs/
| storage/
|
| .env.example
| docker-compose.yml
| README.md
| Makefile
| pyproject.toml
└─ package.json
```

### 3. Frontend Structure

The frontend contains the Antigravity-style workspace UI.

```
frontend/
|
| └─ public/
|
| └─ src/
|   |
|   | └─ app/
|   | └─ routes/
|   | └─ layouts/
|   | └─ pages/
|   |
|   | └─ components/
|   |   | └─ browser/
|   |   | └─ terminal/
|   |   | └─ code-editor/
|   |   | └─ chat/
|   |   | └─ timeline/
|   |   | └─ sidebar/
|   |   | └─ memory/
|   |   | └─ tools/
|   |   | └─ verification/
|   |   └─ common/
|   |
|   | └─ hooks/
|   | └─ contexts/
|   | └─ stores/
|   └─ services/
```

```
|
|   ├── api/
|   ├── utils/
|   ├── assets/
|   ├── styles/
|   └── types/
|
|   ├── tests/
|   └── package.json
```

---

## 4. Backend Structure

The backend hosts the orchestration platform.

```
backend/
|
|   ├── api/
|   ├── orchestrator/
|   ├── planner/
|   ├── reasoner/
|   ├── verifier/
|   ├── execution-controller/
|   ├── task-manager/
|   ├── memory-manager/
|   ├── tool-manager/
|   ├── repository-manager/
|   ├── event-manager/
|   ├── auth/
|   ├── monitoring/
|   ├── reporting/
|   ├── websocket/
|   ├── middleware/
|   ├── services/
|   ├── utils/
|   ├── models/
|   ├── schemas/
|   └── tests/
```

---

## 5. AI Layer Structure

The AI layer contains model-specific logic.

```
ai/
|
```

```
|─ planner/
|   |─ prompts/
|   |─ workflows/
|   |─ strategies/
|   └─ planners/
|
|─ reasoner/
|   |─ prompts/
|   |─ analyzers/
|   |─ patch-generation/
|   |─ root-cause/
|   └─ reasoning/
|
|─ verifier/
|   |─ prompts/
|   |─ validators/
|   |─ reviewers/
|   └─ regression-analysis/
|
|─ providers/
|   |─ gemini/
|   |─ openai/
|   |─ anthropic/
|   └─ adapters/
|
└─ shared/
```

---

## 6. Orchestrator Structure

The orchestrator is the brain of execution.

```
backend/orchestrator/
|
|─ engine/
|─ workflows/
|─ task-routing/
|─ state-machine/
|─ task-state/
|─ planning/
|─ execution/
|─ recovery/
|─ checkpoints/
|─ schedulers/
└─ policies/
```

## 7. Planner Structure

```
backend/planner/  
|  
├─ goal-analysis/  
├─ decomposition/  
├─ task-generation/  
├─ prioritization/  
├─ dependency-resolution/  
├─ workflow-generation/  
├─ planning-memory/  
└─ plan-validation/
```

---

## 8. Reasoner Structure

```
backend/reasoner/  
|  
├─ code-analysis/  
├─ architecture-analysis/  
├─ root-cause/  
├─ bug-analysis/  
├─ security-analysis/  
├─ patch-generation/  
├─ impact-analysis/  
├─ compatibility-analysis/  
└─ reasoning-engine/
```

---

## 9. Verifier Structure

```
backend/verifier/  
|  
├─ evidence-review/  
├─ patch-review/  
├─ regression-review/  
├─ test-validation/  
├─ finding-validation/  
├─ confidence-analysis/  
└─ decision-engine/
```

---

## 10. Memory System Structure

```
memory/  
|  
├─ working-memory/  
├─ session-memory/  
├─ project-memory/  
├─ experience-memory/  
|  
├─ retrieval/  
├─ ranking/  
├─ summarization/  
├─ compression/  
├─ embeddings/  
├─ indexing/  
├─ persistence/  
└─ memory-policies/
```

---

## 11. Repository Analysis Structure

```
repository-analysis/  
|  
├─ ingestion/  
├─ indexing/  
├─ ast/  
├─ dependency-graph/  
├─ call-graph/  
├─ symbol-analysis/  
├─ file-ranking/  
├─ chunking/  
├─ embeddings/  
├─ static-analysis/  
└─ candidate-extraction/
```

---

## 12. Tool Layer Structure

The tool layer should remain independent from reasoning.

```
tools/  
|  
├─ terminal/
```

```
| browser/  
| filesystem/  
| search/  
| code-runner/  
| git/  
| github/  
| parsers/  
| adapters/  
| sandbox/  
└─ tool-registry/
```

---

## 13. Browser Tool Structure

```
tools/browser/  
|  
├─ navigation/  
├─ rendering/  
├─ inspection/  
├─ screenshots/  
├─ dom-analysis/  
├─ page-state/  
├─ automation/  
└─ browser-controller/
```

This supports the embedded browser workspace.

---

## 14. Terminal Tool Structure

```
tools/terminal/  
|  
├─ command-runner/  
├─ shell-manager/  
├─ process-manager/  
├─ output-parser/  
├─ session-manager/  
├─ logs/  
└─ terminal-controller/
```

---

## 15. Execution Layer Structure

Execution should remain isolated.

```
execution/
|
├─ sandboxes/
├─ runners/
├─ execution-jobs/
├─ task-execution/
├─ process-control/
├─ permissions/
├─ environments/
├─ recovery/
└─ monitoring/
```

---

## 16. Event System Structure

```
backend/event-manager/
|
├─ publishers/
├─ consumers/
├─ event-types/
├─ handlers/
├─ persistence/
├─ replay/
└─ analytics/
```

---

## 17. Reporting Structure

```
backend/reporting/
|
├─ reports/
├─ findings/
├─ patches/
├─ summaries/
├─ verification/
├─ exports/
└─ templates/
```

---



## 18. Authentication Structure

```
backend/auth/  
|  
├─ login/  
├─ registration/  
├─ sessions/  
├─ permissions/  
├─ roles/  
├─ tokens/  
└─ providers/
```

---

## 19. API Structure

```
backend/api/  
|  
├─ v1/  
|   |  
|   ├── auth/  
|   ├── tasks/  
|   ├── agents/  
|   ├── memory/  
|   ├── repositories/  
|   ├── tools/  
|   ├── reports/  
|   ├── settings/  
|   └─ users/  
└─ middleware/
```

---

## 20. WebSocket Structure

Real-time updates.

```
backend/websocket/  
|  
├─ task-streams/  
├─ terminal-streams/  
├─ browser-streams/  
└─ notifications/
```

```
| progress/  
└─ session-events/
```

---

## 21. Database Layer Structure

The actual schemas will be defined later.

```
storage/  
|  
├─ postgres/  
|  
├─ vector-db/  
|  
├─ object-storage/  
|  
├─ cache/  
|  
└─ backups/
```

---

## 22. Infrastructure Structure

```
infrastructure/  
|  
├─ docker/  
├─ kubernetes/  
├─ monitoring/  
├─ logging/  
├─ tracing/  
├─ networking/  
├─ secrets/  
├─ ci-cd/  
└─ security/
```

---

## 23. Monitoring Structure

```
backend/monitoring/  
|  
├─ metrics/  
└─ traces/
```

```
├─ logs/
├─ alerts/
├─ dashboards/
└─ health-checks/
```

---

## 24. Shared Module Structure

Shared code should live separately.

```
shared/
|
├─ constants/
├─ enums/
├─ interfaces/
├─ types/
├─ helpers/
├─ validators/
├─ events/
└─ exceptions/
```

---

## 25. Configuration Structure

```
configs/
|
├─ development/
├─ staging/
├─ production/
|
├─ ai/
├─ tools/
├─ memory/
├─ execution/
└─ security/
```

---

## 26. Testing Structure

```
tests/
|
├─ unit/
```

```
| integration/
| e2e/
| performance/
| security/
| regression/
└─ fixtures/
```

---

## 27. Documentation Structure

```
docs/
|
|─ product/
|─ architecture/
|─ api/
|─ deployment/
|─ operations/
|─ security/
|─ development/
└─ decisions/
```

---

## 28. Deployment Structure

```
deployments/
|
|─ local/
|─ development/
|─ staging/
|─ production/
└─ disaster-recovery/
```

---

## 29. Logging Structure

```
logs/
|
|─ api/
|─ agents/
|─ execution/
|─ tools/
|─ memory/
```

```
|— repositories/  
|— system/
```

---

## 30. Recommended Development Order

Build sequence:

1. Shared
2. Backend Core
3. API
4. Planner
5. Memory
6. Tool Layer
7. Repository Analysis
8. Reasoner
9. Verifier
10. Execution Controller
11. Frontend
12. Monitoring
13. Deployment

This minimizes rework and dependency conflicts.

---

## 31. Folder Structure Principles

The structure follows:

- Separation of concerns
- Independent scalability
- Independent deployment
- Clear ownership
- Testability
- Future extensibility

Each major subsystem has a dedicated boundary.

---

## 32. Final Folder Structure Summary

This structure is optimized for:

- Three-LLM architecture
- Long-horizon execution
- Tool orchestration

- Memory-centric workflows
- Repository intelligence
- Verification pipelines
- Embedded browser workspace
- Embedded terminal workspace
- Antigravity-style execution UI
- Future multi-agent expansion

The repository is intentionally organized as a modular production system rather than a monolithic AI application.